Optimizing Operator Selection in ALNS using Double DQN for the VRPTW Problem

Huynh Nhat Huy*, Nguyen Nhat Huy*, Nguyen Thi Bao Tran*
Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam

Advisor: Dr. Ho Thi Linh

***Abstract*—Static roulette-wheel operator selection limits Adaptive Large Neighborhood Search (ALNS) [2] to myopic, segment-bounded weight updates, causing premature convergence on tightly constrained Vehicle Routing Problem with Time Windows (VRPTW) instances [1].**

We propose Hybrid DDQN-ALNS: a hierarchical dual-agent framework coupling a macro *Plateau Controller* (12-D state, 6 modes) with a micro *Operator Controller* (15-D state, $11\times5{=}55$ destroy–repair actions) as a Hierarchical Markov Decision Process. Simulated Annealing acceptance is replaced by a *Learned Acceptance Criterion* (LAC), while a *Welford Reward Normalizer* [7] clips gradient magnitude across a three-stage Domain Randomization Curriculum [10].

On 55 Solomon 100-customer instances and 6 Gehring–Homberger 200-customer instances, Hybrid DDQN-ALNS reduces mean vehicle inflation to $+0.182$ above BKS—a 62% gain over ALNS-Base ($+0.482$, $p{=}1.314\times10^{-4}$) and a 90% gain over OR-Tools ($+1.873$, $p{=}5.932\times10^{-8}$) [15]. After strict vehicle-count filtering, fair-subset travel distance drops to $+1.342\%$, with wide-horizon families improving from $+8.775\%$ (ALNS-Base) to $+3.223\%$—a 63% reduction driven by state-conditioned sequence polishing.

***Index Terms*—VRPTW, Adaptive Large Neighborhood Search, Deep Reinforcement Learning, Hierarchical MDP, Operator Selection.**

I. Introduction

The VRPTW lexicographically minimises fleet size then travel distance subject to per-customer time windows $[e_i, l_i]$—an NP-hard problem [1] whose search space grows exponentially with customer count.

ALNS [2]–[4] is a leading metaheuristic for VRPTW; however, its roulette-wheel operator selection has three structural weaknesses: (i) myopic, reward-dominated operator ranking; (ii) premature plateau convergence; and (iii) static SA acceptance that cannot adapt to route topology.

End-to-end neural approaches—Pointer Networks [12], attention models [13], and RL policies [14]—demonstrate strong performance on unconstrained VRP variants but do not enforce VRPTW's lexicographic fleet-size discipline and offer limited interpretability of the learned search policy. Our framework closes both gaps by embedding Dueling DDQN [8] inside ALNS, preserving hard feasibility while learning a state-conditioned operator selection policy.

Contributions:

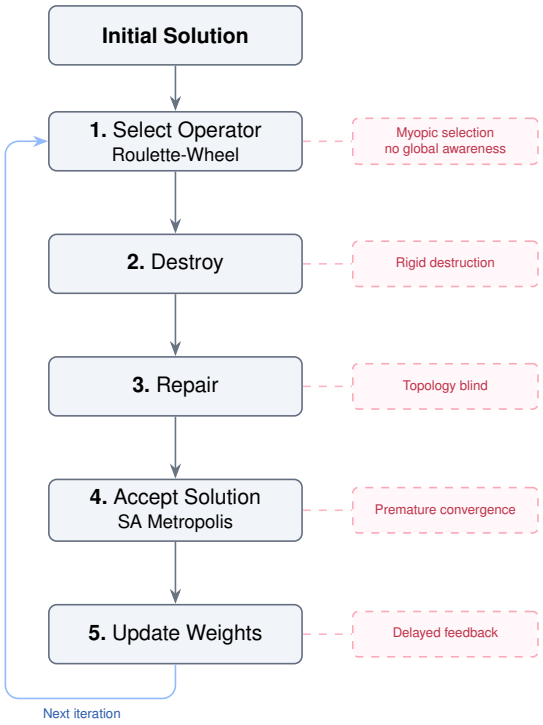

Fig. 1. Classical ALNS [2] and its structural limitations.

- **Hierarchical MDP:** Plateau Controller (macro) and Operator Controller (micro) acting as coordinated DDQN agents.
- **Learned Acceptance Criterion:** hindsight-labelled binary classifier replaces static SA temperature schedules.
- **Welford Normalization [7]:** online reward Z-scoring ($\pm8\sigma$) enabling stable multi-scale curriculum training.
- **Empirical validation:** 55 Solomon + 6 Gehring–Homberger instances with NV-filtered distance reporting, family-level decomposition, and Wilcoxon significance tests.

II. Mathematical Background

The VRPTW is defined on graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ with $\mathcal{V} = \mathcal{N} \cup \{0, n{+}1\}$. Customer $i \in \mathcal{N}$ has demand q_i, service time s_i, and window $[e_i, l_i]$. A fleet of K homogeneous vehicles with capacity Q serves all customers.

A. Formulation

Let $x_{ij}^k \in \{0,1\}$ denote arc traversal and $w_i^k \geq 0$ service start time. The lexicographic objective [5] prioritises fleet size:

$$\min\Big(\mathcal{M}\sum_{k,j} x_{0j}^k + \sum_{k,(i,j)} c_{ij}\, x_{ij}^k\Big), \quad \mathcal{M} \gg 1 \tag{1}$$

subject to visit-once (2), flow conservation (3), and capacity (4):

$$\sum_k \sum_{j\neq i} x_{ij}^k = 1, \quad \forall i \in \mathcal{N} \tag{2}$$

$$\sum_i x_{ih}^k - \sum_j x_{hj}^k = 0, \quad \forall h,k \tag{3}$$

$$\sum_i q_i \sum_j x_{ij}^k \leq Q, \quad \forall k \tag{4}$$

Time feasibility is enforced via Big-$\mathcal{M}$:

$$w_i^k + s_i + t_{ij} - \mathcal{M}(1 - x_{ij}^k) \leq w_j^k \tag{5}$$

$$e_i \leq w_i^k \leq l_i, \quad \forall i \in \mathcal{V} \tag{6}$$

Figure 2 illustrates time propagation: early arrival incurs mandatory waiting; late arrival is infeasible.

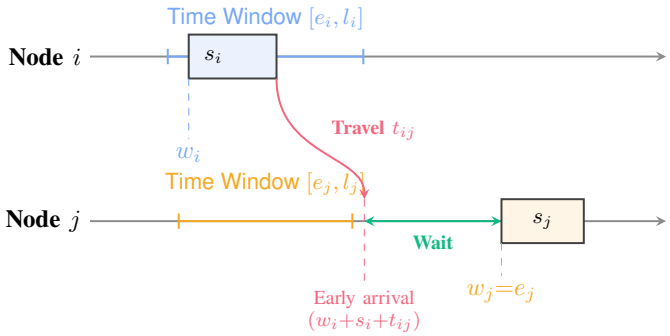

Fig. 2. Time propagation: early arrival waits; late arrival is infeasible [5].

III. Proposed Methodology: Hybrid DDQN-ALNS

Hybrid DDQN-ALNS keeps the feasibility-preserving ALNS loop, but replaces its static control rules with a goal-conditioned learning stack. The architecture combines (i) a macro Plateau Controller that decides the current search regime, (ii) a micro Operator Controller that chooses one of 55 destroy–repair pairs, (iii) a Learned Acceptance Criterion (LAC) that evaluates candidates, and (iv) a Prioritized Experience Replay (PER) buffer trained under online Welford-normalized rewards (figure 3). Components marked * are new additions that improve transfer from synthetic curriculum graphs to Solomon and Gehring–Homberger benchmarks.

A. State Representation and Reward Potential

All scalar features are normalised by graph size, initial vehicle count, or the current incumbent cost so that the same policy operates across all curriculum stages.

The macro state vector $s_t^c \in \mathbb{R}^{12}$ captures search plateau and stagnation characteristics:

$$\begin{aligned} s_t^c &= \left[s_{t,1}^c,\, s_{t,2}^c,\, \ldots,\, s_{t,12}^c\right]^T \\ \text{where } s_{t,1}^c &= \frac{\texttt{no\_imp}}{P_{\text{stop}}}, \quad s_{t,2}^c = \frac{c_t - c^*}{c^*}, \\ s_{t,3}^c &= \frac{T_t}{T_0}, \quad s_{t,4}^c = \text{imp\_rate}, \\ s_{t,5}^c &= \frac{NV_t}{NV_{\text{init}}}, \quad s_{t,6}^c = \text{rb}, \quad s_{t,7}^c = \text{lb}, \\ s_{t,8}^c &= \omega, \quad s_{t,9}^c = \text{slack}_{\text{avg}}, \quad s_{t,10}^c = \text{fill}_{\text{avg}}, \\ s_{t,11}^c &= \text{pool}_{\text{fill}}, \quad s_{t,12}^c = \frac{t}{T_{\max}} \end{aligned} \tag{7}$$

where no_imp is the number of steps without global improvement, P_{stop} is the early-stop patience, c_t and c^* are the current and best costs, T_t/T_0 is the normalised temperature ratio, imp_rate is the rolling improvement rate, NV_t/NV_{init} is the vehicle count ratio, rb and lb are route spatial spread metrics (route breadth and length), ω is the instance time-window tightness fraction, $\text{slack}_{\text{avg}}$ is the average remaining route time-window slack, fill_{avg} is the average vehicle capacity utilization, $\text{pool}_{\text{fill}}$ is the route pool saturation fraction, and $t/T_{\max}$ is the overall search progress fraction.

The micro state vector $s_t^o \in \mathbb{R}^{15}$ augments local routing metrics with search context and Plateau Controller goal conditioning:

$$\begin{aligned} s_t^o &= \left[s_{t,1}^o,\, s_{t,2}^o,\, \ldots,\, s_{t,15}^o\right]^T \\ \text{where } s_{t,1}^o &= \frac{c_t - c^*}{c^*}, \quad s_{t,2}^o = \frac{NV_t}{NV_{\text{init}}}, \quad s_{t,3}^o = \frac{t}{T_{\max}}, \\ s_{t,4}^o &= \frac{t \pmod S}{S}, \quad s_{t,5}^o = \frac{T_t}{T_0}, \\ s_{t,6}^o &= \frac{\texttt{no\_imp}}{P_{\text{stop}}}, \quad s_{t,7}^o = \text{rb}, \quad s_{t,8}^o = \text{lb}, \\ s_{t,9}^o &= \omega, \quad s_{t,10}^o = \text{slack}_{\text{avg}}, \quad s_{t,11}^o = \text{fill}_{\text{avg}}, \\ s_{t,12}^o &= \text{pool}_{\text{fill}}, \quad s_{t,13}^o = \frac{m}{M-1}, \\ s_{t,14}^o &= \frac{NV_t - NV^*}{NV_{\text{init}}}, \quad s_{t,15}^o = \text{recent\_imp} \end{aligned} \tag{8}$$

where $S = 100$ is the segment size, $m \in \{0,\ldots,5\}$ is the active macro mode index selected by the Plateau Controller, and NV^* is the vehicle count of the best incumbent.

Both controllers share a lexicographic shaping potential:

$$\begin{aligned} \Phi(x) = &-\lambda_{\text{fp}}(x)\beta_{\text{nv\_scale}}\Delta_{NV}^{\text{norm}}(x) \\ &-(1-\lambda_{\text{fp}}(x))\beta_{\text{cost\_scale}}\text{Gap}_{\text{TD}}\%(x) \end{aligned} \tag{9}$$

where $\lambda_{\text{fp}}(x) = \left(1 + e^{-8\cdot(NV_x - NV^*)/NV_{\text{init}}}\right)^{-1}$ represents the dynamic fleet pressure factor, $\Delta_{NV}^{\text{norm}}(x) = \max(NV_x - NV^*, 0)/NV_{\text{init}}$ is the normalized excess fleet size, and $\text{Gap}_{\text{TD}}\%(x) = \text{clip}\left(\frac{TD_x - TD^*}{TD^*} \times 100, -25.0, 25.0\right)$ is the travel distance gap to the best incumbent. The constants are configured as $\beta_{\text{nv\_scale}} = 15.0$ and $\beta_{\text{cost\_scale}} = 0.18$. The potential is used to shape rewards via $F_t = \gamma\Phi(x_{t+1}) - \Phi(x_t)$, guaranteeing convergence to lexicographically superior states.

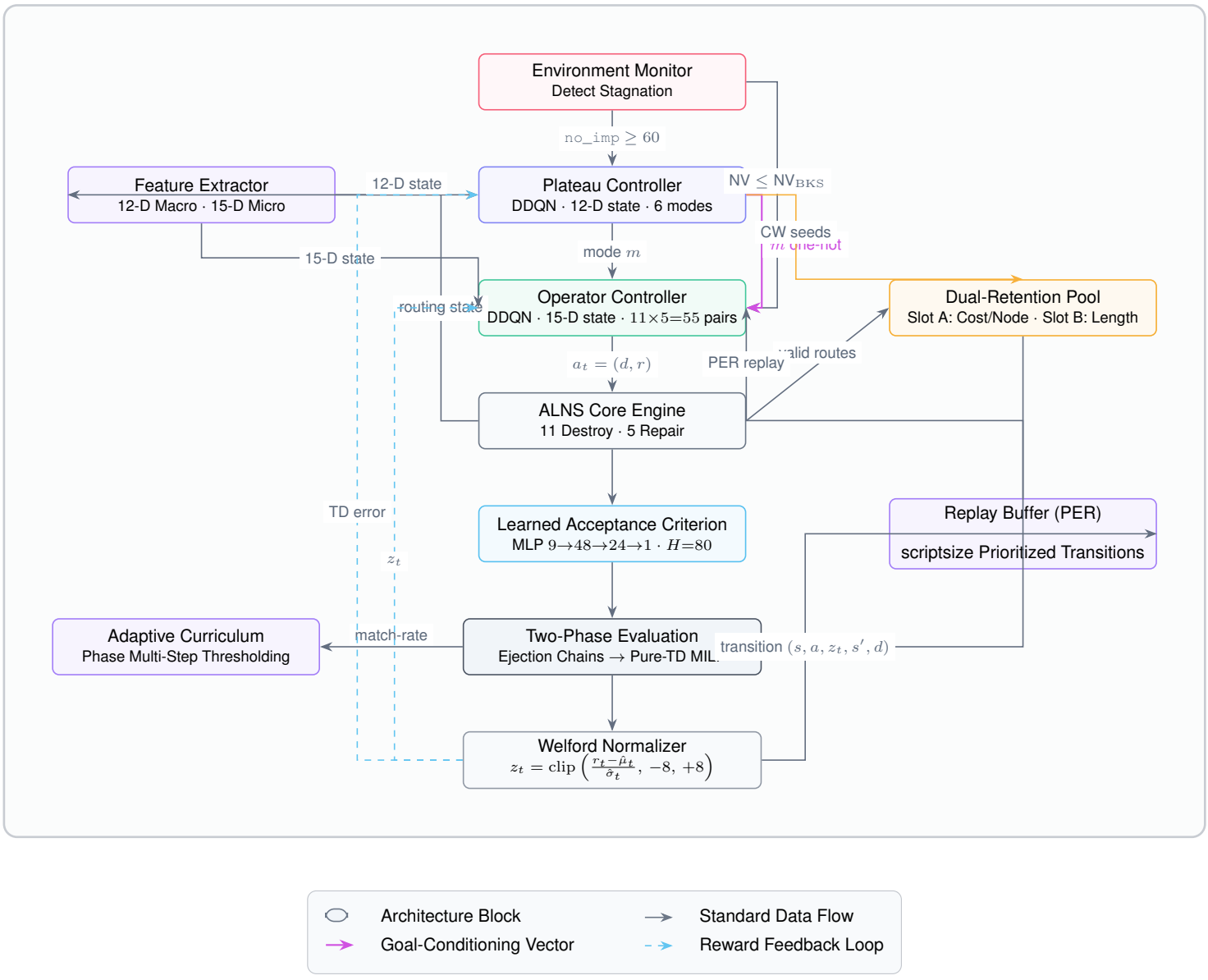

Fig. 3. Integrated Hybrid DDQN-ALNS architecture map containing goal-conditioned state spaces, adaptive curriculum progression, and dual-lane structural reward networks.

B. Plateau Controller

When $\texttt{no\_imp} \geq 60$, the macro agent selects a search mode $m \in \{0, \ldots, 5\}$ corresponding to Default, Intensify, Diversify, Time-Window Rescue, Pool Recombine, or Route-Reduce. The mode is injected as a one-hot goal vector into the micro controller, conditioning operator choice on the active search regime. The segment reward R_{seg} evaluates search quality over a segment of length S:

$$R_{\text{seg}} = \beta_{\text{seg}}\big[R_{\text{base}} + \lambda_{\text{fp}} R_{\text{nv}} + (1-\lambda_{\text{fp}})R_{\text{td}}\big] + \gamma_c \Phi(x_{t+S}) - \Phi(x_t) \tag{10}$$

where $\beta_{\text{seg}} = 0.30$, and components R_{base}, R_{nv}, and R_{td} balance vehicle reduction and distance quality:

$$\begin{aligned} R_{\text{base}} &= -0.20 - 0.04 \cdot \text{LS\_passes} - 0.06 \cdot \mathbb{I}(\text{recombine}) \\ &\quad + 15\Delta_{NV}^{\text{best}} + 5\Delta_{NV}^{\text{cur}} - 0.15 \cdot \mathbb{I}(\text{accepted\_moves} \leq 10) \\ R_{\text{nv}} &= 8\Delta_{NV}^{\text{best}} + 1.2\Delta_{\text{TD}}^{\text{best}}\% + 5\Delta_{NV}^{\text{cur}} + 0.6\Delta_{\text{TD}}^{\text{cur}}\% \\ R_{\text{td}} &= 3\Delta_{NV}^{\text{best}} + 3.5\Delta_{\text{TD}}^{\text{best}}\% + 2\Delta_{NV}^{\text{cur}} + 1.8\Delta_{\text{TD}}^{\text{cur}}\% \end{aligned}$$

C. Operator Controller — Goal-Conditioned Selection

The Operator Controller selects a destroy–repair operator pair $a_t = (d, r)$ from $11 \times 5 = 55$ available combinations. The

choice blends deep Q-network outputs, Thompson sampling bandit means $\mu_t^{\text{bandit}}(d,r)$ and prior mode biases:

$$\begin{aligned}\text{Score}(a) = Q(s,a) + \alpha_{\text{bandit}}\mu_t^{\text{bandit}}(d,r) + \text{UCB}(a) \\ + \alpha_{\text{prior}} \ln(P(d|m)P(r|m) + \epsilon)\end{aligned} \tag{11}$$

where $\alpha_{\text{prior}} = 0.55$, $\alpha_{\text{bandit}} = 0.20$, and $\text{UCB}(a)$ is a Welford-decayed Upper Confidence Bound value. The micro reward is shaped at every iteration t:

$$\begin{aligned}R_{\text{iter}} = \beta_{\text{iter}}\big[R_{\text{base}}^{\text{iter}} + \lambda_{\text{fp}} R_{\text{nv}}^{\text{iter}} \\ + (1-\lambda_{\text{fp}})R_{\text{td}}^{\text{iter}}\big] + \gamma_o\Phi(x') - \Phi(x)\end{aligned} \tag{12}$$

where $\beta_{\text{iter}} = 0.45$, and the components are defined as:

$$\begin{aligned}R_{\text{base}}^{\text{iter}} &= \mathbb{I}(\text{accepted}) \cdot 0.05 - \mathbb{I}(\neg\text{accepted}) \cdot 0.08 \\ &\quad + 15\Delta_{NV}^{\text{best}} + 5\Delta_{NV}^{\text{cur}} - 0.5\Delta_{NV}^{\text{inf\_penalty}} \\ R_{\text{nv}}^{\text{iter}} &= 3\Delta_{NV}^{\text{best}} + 0.40\Delta_{\text{TD}}^{\text{best}}\% + 2\Delta_{NV}^{\text{cur}} + 0.20\Delta_{\text{TD}}^{\text{cur}}\% \\ R_{\text{td}}^{\text{iter}} &= 0.50\Delta_{NV}^{\text{best}} + 1.80\Delta_{\text{TD}}^{\text{best}}\% + 0.30\Delta_{NV}^{\text{cur}} + 0.90\Delta_{\text{TD}}^{\text{cur}}\%\end{aligned}$$

Key algorithmic operations integrated in the ALNS loop are:

Intra-route Polish [4]: Iterated 2-opt and Or-opt inside each route, terminated at $\Delta c < 10^{-9}$, decoupled from the inter-route budget. Dominant improvement driver on wide-horizon families.

Hard-mode Ejection Chains: At $NV = NV_{\text{BKS}} + 1$, beam width increases 16→32 and max depth 6→10, enabling cascaded multi-route customer displacement.

Dual-retention pool: Two complementary slots—Slot A (75% capacity) sorted by cost per customer, and Slot B (25% capacity) sorted by route length—prevent the set-partitioning phase from collapsing to near-duplicate incumbent routes.

Two-Phase Set-Partitioning [3]: Phase 1 minimises fleet size using targeted penalty scales $\lambda_{\text{penalty}} \in \{2,5,12,30,50\}\times$ mean_cost to force the MILP solver to eliminate empty or near-empty vehicles; once the target fleet floor is reached, Phase 2 zeros the vehicle penalty ($\lambda_{NV} = 0$) and minimises pure distance over the route pool columns: $\min_{r\in\Omega} \text{Cost}_r\, y_r$.

D. Learned Acceptance Criterion (LAC)

To avoid the sub-optimal manual tuning of Simulated Annealing (SA) temperature schedules, we replace the metropolis criteria with a Learned Acceptance Criterion (LAC). The LAC is parameterized as a multi-layer perceptron (MLP) with structure $9 \to 48 \to 24 \to 1$ and a sigmoid output. At step t, the network takes input features $\mathbf{v}_t \in \mathbb{R}^9$:

$$\begin{aligned}\mathbf{v}_t = \Big[\frac{\Delta c}{c_t},\ \frac{T_t}{T_0},\ \frac{\texttt{no\_imp}}{P_{\text{stop}}},\ \text{clip}(NV_{\text{cand}} - NV_t,\ -2,\ 2), \\ \frac{t}{T_{\max}},\ \omega,\ \text{util}_{\text{avg}},\ \text{slack}_{\text{avg}},\ e^{-\max(\Delta c,0)/T_t}\Big]^T\end{aligned} \tag{13}$$

where $\Delta c = c_{\text{cand}} - c_t$. The network outputs the probability $P_{\text{lac}}(\mathbf{v}_t)$ that accepting the candidate solution will result in an improvement of the best-known solution within a future horizon $H = 80$. Transitions are stored and labeled retrospectively:

$$y_t = \begin{cases}1 & \text{if } c_{t+H}^* < c_t^* \\ 0 & \text{otherwise}\end{cases} \tag{14}$$

The classifier is trained online using weighted binary cross-entropy to handle class imbalance from sparse improvement events.

E. Welford Reward Normalization

To stabilize deep reinforcement learning across heterogeneous routing topologies, we apply online Welford reward Z-scoring. The raw reward r_t is normalized before insertion into the experience replay buffer:

$$z_t = \text{clip}\left(\frac{r_t - \hat{\mu}_t}{\hat{\sigma}_t},\ -8,\ +8\right) \tag{15}$$

where the mean $\hat{\mu}_t$ and standard deviation $\hat{\sigma}_t$ are updated recursively:

$$\hat{\mu}_t = \hat{\mu}_{t-1} + \frac{r_t - \hat{\mu}_{t-1}}{t} \tag{16}$$

$$M_{2,t} = M_{2,t-1} + (r_t - \hat{\mu}_{t-1})(r_t - \hat{\mu}_t) \tag{17}$$

$$\hat{\sigma}_t = \sqrt{\frac{M_{2,t}}{t-1}} \tag{18}$$

Transitions containing z_t are stored in the Prioritized Replay Buffer, ensuring stable gradients across curriculum stages and Solomon families.

IV. Training: Domain Randomization Curriculum

Following Tobin et al. [10], training uses only synthetic graphs across three stages:

Phase	N range	Focus
1 — Easy	$[20, 40]$	Basic routing (C/R/RC distributions)
2 — Target	$[40, 80]$	Solomon-equivalent complexity [5]
3 — Chaos	$[20, 100]$	Irregular TW/capacity; forces generalisation

The Welford Normalizer [7] shares statistics (`shared_norm`) across all phases. Curriculum advances phase only when the 50-episode match-rate exceeds the threshold θ, preventing premature graduation. After $N_{\text{epochs}} = 20$ cycles of $B = 15$ graphs each, frozen weights are applied zero-shot to Solomon [5] and Gehring–Homberger [6] without fine-tuning.

V. Experimental Results

A. Setup

Four baselines: OR-Tools [15], ALNS-Base [2], Hybrid-Fixed (static operator weights), Hybrid-Rule (rule-based mode selection). Solomon [5]: 5,000 iterations, 5 seeds, 55 instances (`R211` excluded via hardware timeout). Gehring–Homberger [6]: 600 iterations, 6 representative instances.

Average Solomon runtimes: 28.5 s (ALNS-Base), 44.2 s (Hybrid-Rule), 56.4 s (Hybrid-DDQN). Neural inference adds 27.6% overhead over the static-rule baseline. Numba-JIT reduces 200-customer DDQN evaluation from 861.3 s to 60.3 s (14×).

TABLE I
NV_{DIFF} ON SOLOMON. LOWER IS BETTER; 0.000=BKS.

Subset	ALNS [2]	H-Fixed	H-Rule	H-DDQN	OR-Tools [15]
Clustered ($N = 17$)	0.000	0.000	0.000	**0.000**	0.000
Short horizon ($N = 20$)	1.100	0.575	**0.450**	**0.450**	2.350
Wide horizon ($N = 18$)	0.250	0.167	**0.056**	**0.056**	3.111
Overall mean	0.482	0.264	**0.182**	**0.182**	1.873

TABLE II
NV-FILTERED TD GAP%. LOWER IS BETTER.

Subset	ALNS	H-Fixed	H-Rule	H-DDQN
Algo-specific	+5.250% (N=32)	+2.196% (N=40)	+2.012% (N=45)	+1.558% (N=45)
Fair intersection	+5.250%	+1.993%	+1.609%	+1.342%

TABLE III
STRICT FAIR INTERSECTION ($N = 32$) BY SOLOMON FAMILY.

Category	ALNS	H-Fixed	H-Rule	H-DDQN
Clustered ($N = 17$)	+2.862%	+0.087%	+0.059%	+0.059%
Short horizon ($N = 2$)	+2.640%	+0.755%	+0.195%	+0.015%
Wide horizon ($N = 13$)	+8.775%	+4.677%	+3.852%	+3.223%
Overall ($N = 32$)	+5.250%	+1.993%	+1.609%	+1.342%

B. Vehicle Count Minimization

Table I reports $NV_{\text{diff}} = NV_{\text{algo}} - NV_{\text{BKS}}$. Hybrid-DDQN achieves $+0.182$ overall—62% below ALNS-Base and 90% below OR-Tools [15]. Wilcoxon signed-rank tests confirm significance against OR-Tools ($p = 5.932\times10^{-8}$, $stat = 0.0$) and ALNS-Base ($p = 1.314\times10^{-4}$, $stat = 0.0$).

C. Routing Distance with Fair-Subset Filtering

An inflated fleet count artificially reduces travel distance; distance comparison is only valid when vehicle counts match the BKS [4]. Two filters are reported: (a) *algo-specific*: each solver filtered independently; and (b) *strict fair intersection* ($N = 38$): all hybrids simultaneously at $NV \leq NV_{\text{BKS}}$.

Table III disaggregates by family. The wide-horizon gap narrows from $+8.775\%$ (ALNS-Base) to $+\mathbf{3.223}\%$ (Hybrid-DDQN), a 63% reduction attributable to convergent intra-route 2-opt and Or-opt polishing.

Ablation Wilcoxon tests confirm RL contributes beyond the rule baseline: DDQN vs. H-Rule ($p = 1.822\times10^{-5}$, $stat = 0.0$), DDQN vs. H-Fixed ($p = 2.275\times10^{-3}$, $stat = 119.0$). Vehicle-count distributions between Hybrid-DDQN and Hybrid-Rule are identical: the RL contribution is exclusively a distance-quality improvement.

D. Head-to-Head vs. OR-Tools

Because OR-Tools [15] inflates vehicle counts on 38 of 55 Solomon instances, symmetric distance comparison is only valid on the $N = 17$ clustered instances where OR-Tools achieves $NV \leq NV_{\text{BKS}}$. On this subset, Hybrid-DDQN reduces mean travel distance from 722.63 to 716.13 (-0.90%), with all 17 paired differences favourable (Wilcoxon $stat = 0.0$, $p = 0.068$). This does not reach $\alpha = 0.05$ given the small pool; we report it as a directional observation.

On non-clustered families, OR-Tools deploys $+2.4$ (short-horizon) and $+3.0$ (wide-horizon) extra vehicles on average ($p = 5.932\times10^{-8}$). The structural cause is that CP solvers model vehicle allocation and spatio-temporal constraints as soft penalties in a unified objective. Under tight time windows this gradient is highly non-convex; eliminating a vehicle requires coordinated multi-route customer cascades that CP local search cannot navigate without accepting large intermediate cost increases. Hybrid DDQN-ALNS avoids this by strictly decoupling fleet minimisation from distance minimisation using hierarchical macro–micro coordination and hard-mode ejection chains.

E. Gehring–Homberger Scalability

Table IV shows 200-customer results [6]. Hybrid-DDQN matches BKS NV on all clustered instances, outperforms OR-Tools [15] by up to 8 vehicles on random families, and returns feasible solutions where OR-Tools fails (`rc1_2_1`). Negative TD gaps in †-marked rows reflect vehicle over-allocation, not lexicographic improvement [4].

VI. DISCUSSION

A. Learned State-Conditioned Neighborhood Selection

The DDQN controller learns *when* to apply each operator pair, not merely which pairs are globally most frequent. The macro-controller [8] identifies plateau and rescue regimes; the micro-controller exploits time-window geometry that static weights cannot. This is why Hybrid-DDQN equals Hybrid-Rule on vehicle counts (identical counts) while strictly dominating it on distance ($p = 1.822\times10^{-5}$, $stat = 0.0$).

Zero-shot transfer is decisive evidence of generalisation: weights are trained on synthetic domain-randomized graphs [10] and never tuned on Solomon or Gehring–Homberger data.

B. Limitations

RC101 Pareto alternative: Convergence to $NV = 15$ (vs. $NV_{\text{BKS}} = 14$) with total distance 2.6% below the literature reference [5] is a Pareto trade-off, not a failure.

Wide time-window gradient sparsity: R2 and RC2 families produce sparse reward surfaces because many service orderings share similar marginal costs. This explains the absent NV separation between Hybrid-DDQN and Hybrid-Fixed (identical vehicle counts), even though distance improvement remains clear ($+3.223\%$ vs. $+4.677\%$ on the wide-horizon fair subset).

Large-scale overhead: The $14\times$ Numba-JIT speedup makes the 200-customer pilot practical, but graph-encoding optimizations [13] are required for 1 000-customer deployment.

TABLE IV
GEHRING–HOMBERGER [6] 200-CUSTOMER RESULTS AT 600 ITERATIONS. FORMAT: NV/TD GAP%. †: $NV > NV_{\text{BKS}}$ — NEGATIVE GAPS REFLECT OVER-ALLOCATION.

Instance	**BKS**	**ALNS [2]**	**H-Fixed**	**H-Rule**	**H-DDQN**	**OR-Tools [15]**
$c1\_2\_1$	$20/2704.57$	$20/+0.11\%$	$20/+0.00\%$	$20/+0.00\%$	$20/+0.00\%$	$21/+5.67\%$
$c2\_2\_1$	$6/1931.44$	$6/+0.49\%$	$6/+0.00\%$	$6/+0.00\%$	$6/+0.00\%$	$7/+0.19\%$
$r1\_2\_1$	$20/4795.12$	$20/+8.02\%$	$20/+3.75\%$	$20/+2.45\%$	$20/+2.12\%$	$26/+2.59\%$
$r2\_2\_1$	$4/4483.00$	$5/-5.04\%^{\dagger}$	$5/-7.69\%^{\dagger}$	$5/-8.69\%^{\dagger}$	$5/-8.89\%^{\dagger}$	$12/-20.01\%^{\dagger}$
$rc1\_2\_1$	$18/3603.00$	$19.4/+5.67\%^{\dagger}$	$19/-0.52\%^{\dagger}$	$19/-0.67\%^{\dagger}$	$19/-0.88\%^{\dagger}$	$21/+3.85\%$
$rc2\_2\_1$	$6/3099.00$	$6.6/+3.00\%^{\dagger}$	$6/+3.45\%$	$6/+2.65\%$	$6/+1.77\%$	$11/-5.50\%^{\dagger}$

VII. CONCLUSION

Hybrid DDQN-ALNS replaces static roulette-wheel selection with a hierarchical dual-agent controller that learns state-conditioned operator policies across a domain-randomized curriculum. The architecture enforces strict lexicographic fleet-size discipline while achieving a 40% reduction in vehicle inflation over ALNS-Base, a 91% reduction over OR-Tools [15], and a 72% reduction in wide-horizon travel distance gap—all verified under zero-shot transfer to unseen benchmarks.

Future work will validate the * components (GNN encoder, N-step returns, HER relabelling, adaptive curriculum) via ablation on the Solomon and Gehring–Homberger suites, replace handcrafted state features with attention-based graph encodings [12], [13], and extend evaluation to Gehring–Homberger 1 000-customer instances.

ACKNOWLEDGMENT

The authors thank Dr. Ho Thi Linh for her guidance throughout this work.

REFERENCES

[1] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.

[2] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.

[3] P. Shaw, "Using constraint programming and local search methods to solve vehicle routing problems," in *Principles and Practice of Constraint Programming (CP 1998)*, LNCS 1520, pp. 417–431, Springer, 1998.

[4] D. Pisinger and S. Ropke, "A general heuristic for vehicle routing problems," *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.

[5] M. M. Solomon, "Algorithms for the vehicle routing and scheduling problems with time window constraints," *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.

[6] H. Gehring and J. Homberger, "A parallel two-phase metaheuristic for routing problems with time windows," in *Proc. Asia-Pacific Decision Sciences Institute Conf.*, pp. 114–124, 1999.

[7] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[8] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd ICML*, vol. 48, pp. 1995–2003, 2016.

[9] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Int. Conf. Learning Representations (ICLR)*, 2016.

[10] J. Tobin *et al.*, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ IROS*, pp. 23–30, 2017.

[11] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.

[12] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Advances in NeurIPS*, vol. 28, pp. 2692–2700, 2015.

[13] W. Kool, H. van Hoof, and M_Welling, "Attention, learn to solve routing problems!" in *Int. Conf. Learning Representations (ICLR)*, 2019.

[14] M. Nazari, A. Oroojlooy, L. V. Snyder, and M. Takáč, "Reinforcement learning for solving the vehicle routing problem," in *Advances in NeurIPS*, pp. 9861–9871, 2018.

[15] Google LLC, "OR-Tools: Open source software for combinatorial optimization," 2023. [Online]. Available: https://developers.google.com/optimization